ANALYSIS REFERENCE

Nested sampling analysis, step by step

From CP(N-1) dead-point files to stable weighted observables, beta-only reweighting, stream-bootstrap uncertainties, and validated outputs.

The complete pipeline

1. **Discover** run folders
2. **Load** aligned data
3. **Merge** by energy
4. **Weight** in log space
5. **Evaluate** observables
6. **Reweight** beta only
7. **Bootstrap** and export

What this workflow guarantees

- Independent streams are merged with the correct effective live-point count.
- Tiny weights are normalized in log space; signed means retain cancellation accuracy.
- The source-prior mass is not confused with a physical partition function.
- Reweighting changes beta only and keeps theta fixed.
- Uncertainty comes from resampling whole streams, not individual dead points.

Primary implementation

analyze_nested_sampling.py orchestrates the workflow; utils/nest_sampl.py provides combination, stable quadrature, reweighting, and bootstrap primitives.

Canonical source prior

The live points are already sampled at the configured source beta. At that beta, the additional likelihood is 1.

Extensive normalization

From epsilon = dati[:,1], use H = 2 * N * V * epsilon and S_beta = beta * H.

Correlation-aware errors

Bootstrap complete independently seeded streams, re-merge each replicate, and recompute every weight.

STEP 1-2

Discover, validate, and align inputs

The loader turns each run directory into one validated NestedSamplingStream with matched energy and observable arrays.

Input	Used fields	Role in analysis
input.conf	size, beta, theta, num_live_pts, cooling cadence	Defines lattice geometry, source ensemble, and stream live-point count.
dati.dat	action density, G(0), G(k)	Supplies the ordering variable and non-topological observables.
topo.dat	Q_U, Q_z, Q_plaq by cooling step	Provides topological observables at one selected cooling level.
acceptance_rate.dat	row count	Caps the analysis at the last complete iteration when present.

Run discovery

- A positional input may be a run directory or a parent containing immediate run0 / run_0 style children.
- Duplicate resolved paths are removed while preserving input order.
- All selected runs must share one square lattice size, one finite source beta, and one finite fixed theta.
- Every observable array must match the energy array and contain only finite values.

Alignment and cooling selection

Stage	Rule
General measurements	Use complete rows from dati.dat.
Topology blocks	Reshape by num_cool_step / meas_cool_every + 1, then select the requested cooling step.
Common count	Take the minimum of general rows, complete topology blocks, and acceptance rows when present.
Continuation data	Account for repeated checkpoint measurements and reset iteration labels before joining segments.

```
epsilon = dati.dat[:, 1] E_analysis = action_prefactor * epsilon default action_prefactor = 2
```

Compile-time N is external metadata

N is part of the executable and is not stored in input.conf. Supply it explicitly with --cpn-n when reporting extensive quantities or reweighting beta.

STEP 3-4

Merge streams and construct quadrature weights

Independent dead-point sequences become one energy-ordered deterministic quadrature under the combined source prior.

Merge independent streams

- Concatenate all selected dead points and sort them by decreasing energy.
- Apply the same stable ordering to every observable array.
- Set the effective live-point count to the sum across streams: K_eff = sum_s K_s.

$$\log t = -1 / K_{\mathrm{eff}} \quad X_i = t^{i+1} \quad w_i = t^i * (1 - t)$$

Stable log-weight construction

Because t is close to one, the small shell width 1 - t must not be formed by subtracting two nearly equal floating-point numbers. The implementation uses expm1 and keeps normalization in log space.

$$\log(1 - t) = \log(-\mathrm{expm1}(\log t)) \quad \log w_i = i * \log t + \log(1 - t)$$

$$\log M_{\mathrm{dead}} = \mathrm{logsumexp}(\log w) \quad \log w_{\mathrm{norm},i} = \log w_i - \log M_{\mathrm{dead}}$$

Source-prior coverage diagnostics

Output field	Meaning
log_accumulated_prior_mass	log M_dead = log(sum_i w_i); approaches zero when the run covers almost all source-prior mass.
log_remaining_prior_mass	log X_final = final log_x; retain this form when the linear remainder underflows.
remaining_prior_mass	X_final in linear space; useful when it is representable.
log_Z	Deprecated compatibility alias for log_accumulated_prior_mass; not a physical partition function.

Coverage identity

Under the expected-shrinkage convention, accumulated dead mass plus remaining live mass equals one up to floating-point round-off: M_dead + X_final = 1.

STEP 5

Evaluate observables with stable weighted sums

Every reported mean uses normalized nested-sampling quadrature weights. Derived quantities are built from those weighted estimates.

Weighted means

$$\langle O \rangle = \sum_i w_{\mathrm{norm},i} \, O_i$$

Representable products are summed directly to preserve precision when signed observations cancel. Products that underflow to zero are recovered separately with signed log-sum-exp. For non-negative observables, log_mean returns the result entirely in log space.

$$\log \langle O \rangle = \mathrm{logsumexp}(\log w_{\mathrm{norm},i} + \log O_i) \text{ for } O_i \geq 0$$

Centered fluctuations

Variances are evaluated from centered observations instead of subtracting two large moments.

$$\mathrm{Var}(Q) = \sum_i w_{\mathrm{norm},i} \, (Q_i - \,)^2 \quad \chi_{\mathrm{top}} = \mathrm{Var}(Q_U) / L^2$$

Correlation length

$$\xi_L^2 = (G(0) / G(k) - 1) / [4 \sin(\pi / L)^2] \quad \xi_L = \sqrt{\xi_L^2}$$

Action normalization

$$H = 2 N V \varepsilon \quad S_\beta = \beta H \quad V = L^2$$

Reported group	Examples
Source coverage	log_accumulated_prior_mass, log_remaining_prior_mass, remaining_prior_mass
Action	mean_action_density, mean_energy, mean_extensive_energy, mean_symanzik_action
Correlators	mean_G0, mean_Gk, xi_L_squared, xi_L
Topology	mean_Q_U, mean_Q_U_squared, chi_topological, mean_Q_z, mean_Q_plaquette
Reweighting diagnostics	log_Z_ratio and quadrature_effective_sample_size when --target-beta is used

Interpretation rule
Compare with regular MCMC only after weighting and confirming matched beta, initialization, and prior convention.

STEP 6

Reweight beta - and beta only

The source quadrature represents the normalized canonical ensemble at beta_0. A target beta_1 is obtained by a density-ratio factor; theta remains unchanged.

Reweighting factor

```
Delta beta = beta_1 - beta_0 log L_i = -Delta beta * H_i
```

```
log W_i = log w_i - Delta beta * H_i log[Z(beta_1)/Z(beta_0)] = logsumexp(log W)
```

```
log W_norm,i = log W_i - log[Z(beta_1)/Z(beta_0)]
```

What remains fixed

- The action discretization, lattice geometry, cooling choice, and all measured observables.
- Theta: all selected runs must share one finite value, and no target-theta option exists.
- The source-prior X coordinate: only the quadrature density over it is reweighted.

Quadrature concentration diagnostic

```
ESS_quad = 1 / sum_i W_norm,i^2
```

ESS_quad indicates how concentrated the deterministic target weights are. It is not an independent-sample count and does not replace the stream bootstrap. A very small value warns that the target beta depends on a narrow part of the source quadrature.

Required metadata

Pass the executable's compile-time N with --cpn-n. The analysis uses raw action_density, so beta reweighting is independent of the optional plotting/order action-prefactor.

Local command

```
python analyze_nested_sampling.py RUN_ROOT [RUN_ROOT ...] \
    --output analysis_results/reweighted \
    --bootstrap 1000 --seed 1234 \
    --cpn-n 21 --target-beta 0.6001
```

UBELIX wrapper

```
TARGET_BETA=0.6001 sbatch sh_files/analyze_4x5000_ubelix.sh
```

STEP 7

Bootstrap whole streams, export, and verify

Statistical uncertainty is estimated at the level of independently seeded runs, preserving all within-stream correlations.

One bootstrap replicate

- Draw the same number of streams with replacement.
- Concatenate the selected streams, including repeated selections.
- Sort all dead points again by decreasing energy.
- Recompute K_eff, source quadrature weights, and optional beta-reweighted weights.
- Recompute every observable and diagnostic; never resample individual dead points.

```
bootstrap error = sample standard deviation across replicates ddof = 1
```

Deliverables

Artifact	Contents
summary.txt	Compact estimate +/- bootstrap error entries.
results.json	Settings, run metadata, estimates, errors, and deprecated-field mapping.
bootstrap_samples.csv	All replicate values, only with --save-bootstrap-samples.
nested_sampling_weights_z.png	Normalized weight and cumulative normalized-weight curves.
action_energy.png	Stream and merged action-density traces plus energy histograms.
correlation_length.png	G(0), G(k), and the xi_L estimate.
topological_charge.png	Q_U, Q_z, and Q_plaq across source-prior compression.

Verification checklist

- Run Python bytecode compilation.
- Run the complete unittest suite.
- Exercise source and nearby target beta.
- Exercise a small stream bootstrap.
- Render and inspect all four plots.
- Remove smoke outputs; inspect git diff/status.

```
python -m py_compile analyze_nested_sampling.py utils/nest_sampl.py tests/test_nest_sampl.py
python -m unittest discover -s tests -v
```

Bottom line

Stable source-prior quadrature, beta-only log reweighting, and whole-stream bootstrap form one consistent analysis pipeline.